CAPA - Basics

Common Analysis Platform for Artifacts

Static Malware Analysis and Automated Capability Identification

CAPA is an automated static analysis tool developed by the FireEye Mandiant team to identify the capabilities of executable files without executing them. By analyzing Portable Executables (PE), ELF binaries, and .NET modules against a comprehensive rule set, CAPA provides insights into program behaviors such as network communication, file manipulation, and process injection.

This module explores CAPA's integration with the **MITRE ATT&CK** framework and the **Malware Behavior Catalogue (MBC)**. It details the tool's technical namespaces, advanced command-line parameters for verbose reporting, and the use of the **CAPA Web Explorer** for interactive data dissection.

This foundational knowledge allows security professionals to quickly comprehend malicious functionality and strengthen incident response strategies.

This **documentation** and **learning** approach is strongly inspired by the structured, hands-on content provided by **TryHackMe**, which significantly contributed to practical understanding of "CAPA Tool Basics" concepts.

WEIRDO

DOCUMENTED BY

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN

GITHUB

1. Introduction to Static Analysis with CAPA

CAPA bridges the gap between basic file inspection and complex manual reverse engineering. It performs **Static Analysis**, which means it inspects the code without running it, thereby eliminating the risk of compromising the analysis environment.

- **Core Function:** Identifies "capabilities" (what a program is *capable* of doing).
- **Format Support:** Works with PE, ELF, .NET, and shellcode.
- **Knowledge Encapsulation:** Automates the identification of patterns that typically require years of reverse engineering expertise.

2. Command-Line Operations

CAPA is a terminal-based tool. Its effectiveness depends on the level of verbosity requested during the scan.

Command Section: Tool Execution

Parameter	Description	Example Syntax
-h	**Help:** Lists all available parameters.	capa -h
-v	**Verbose:** Provides detailed results and matches.	capa.exe malware.bin -v
-vv	**Very Verbose:** Lists every instruction that matched a rule.	capa.exe malware.bin -vv
-j	**JSON:** Outputs the result in JSON format for external tools.	capa.exe file.bin -j > report.json

WEIRDO

3. Dissecting CAPA Results

CAPA categorizes its findings into distinct blocks to help analysts prioritize their investigation.

A. General Information and MITRE ATT&CK

The first block provides file hashes (MD5, SHA256) and the system architecture. It then maps findings to the **MITRE ATT&CK Framework**.

- **Format:** Tactic :: Technique :: ID (e.g., DEFENSE EVASION :: Obfuscated Files :: T1027).

B. MAEC (Malware Attribute Enumeration and Characterization)

Standardized labels to define the malware's primary role:

- **Launcher:** Exhibits behaviors that trigger specific actions (drops payloads, persistence).
- **Downloader:** Fetches resources/updates from the internet.

C. Malware Behavior Catalogue (MBC)

Unlike MITRE, MBC is tailored specifically for **Malware Analysis**.

- **Objective:** High-level goal (e.g., Anti-Static Analysis).
- **Behavior:** Specific action (e.g., Virtual Machine Detection).
- **Identifier:** Standardized tag (e.g., **B0009** for VM Detection).

4. Namespaces and Capabilities

CAPA uses **Namespaces** to group rules by their purpose. A **Capability** is essentially the name of a matched rule.

Top-Level Namespace (TLN)	Description
anti-analysis	Rules to evade debuggers, sandboxes, or VMs.
data-manipulation	Rules for encoding (**Base64**, **XOR**) or encrypting data.
persistence	Rules for maintaining access (e.g., **schtasks**).
host-interaction	Rules for file system or process manipulation.
nursery	A staging ground for unpolished or "experimental" rules.

WEIRDO

5. Advanced Analysis: CAPA Web Explorer

To handle the massive amount of data generated by -vv (which can exceed 3,000 lines), analysts use the **CAPA Web Explorer**. This allows for interactive searching and filtering of capabilities.

Practical Analysis Workflow

1. **Generate JSON:** capa.exe -j -vv malware.bin > report.json.
2. **Upload:** Use the "Upload from local" button in the Web Explorer interface.
3. **Inspect:** Click on a capability (e.g., "Reference anti-VM strings") to see the exact assembly code or strings that triggered the match.

6. Practical Scenarios and Summary of Tasks

Throughout the module, **I researched** the capabilities of the **cryptbot.bin** file and **found** several critical indicators of compromise. Below is how **I solved** the practical tasks:

General File Identification (Task 3)

- **Work Done: I practiced** reading the pre-processed CAPA reports using PowerShell.
- **Findings: I founded** the SHA256 hash of the file to be ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c.
- **Technique Mapping: I researched** the "Obfuscated Files" result and **found** its MITRE Identifier was **T1027**.

Behavioral Analysis (Task 4 & 6)

- **Work Done: I researched** the **MBC** block to understand the malware's evasion tactics.
- **Detection: I found** the behavior **"Virtual Machine Detection"** which is identified by tag **B0009**.
- **Micro-Behaviors: I practiced** identifying communication methods and **found** that the micro-behavior for HTTP initiation is **C0002**.
- **Rule Matching: I solved** the rule matching challenge by correlating the capability "check HTTP status code" to its YAML source file, **check-http-status-code.yml**.

Deep Analysis via Web Explorer (Task 7)

- **Work Done: I practiced** using the very verbose output (-vv) in JSON format.
- **Interactive Tooling: I solved** the navigation challenge by uploading **cryptbot_vv.json** to the **CAPA Web Explorer**.
- **Regex Identification: I researched** the rule features and **founded** that the capability "reference anti-VM strings targeting VMWare" was triggered by specific strings such as /VMWare/i.

WEIRDO

Conclusion and Cheatsheet

Conclusion

CAPA is a vital utility for the modern defensive toolkit, allowing analysts to rapidly understand a program's potential impact without the overhead of manual reverse engineering. By integrating framework-based results (MITRE and MBC), it provides a standard language for threat hunters. The ability to pivot from a command-line summary to a deep-dive JSON analysis in the Web Explorer ensures that no hidden malicious functionality goes unnoticed.

Fast Revision Cheatsheet

- **CAPA:** Automated static analysis tool by Mandiant.
- **-v / -vv:** Verbose and Very Verbose (for matching code instructions).
- **-j:** JSON output (Required for Web Explorer).
- **Launcher vs. Downloader:** Launcher triggers actions; Downloader fetches files.
- **Namespace:** Logic-based grouping (e.g., anti-analysis for evasion).
- **B0009:** MBC ID for Virtual Machine Detection.
- **T1027:** MITRE ID for Obfuscated Files.
- **Nursery TLN:** The location for experimental or unpolished rules.
- **YAML (.yml):** The file format used to define CAPA's detection rules.

WEIRS